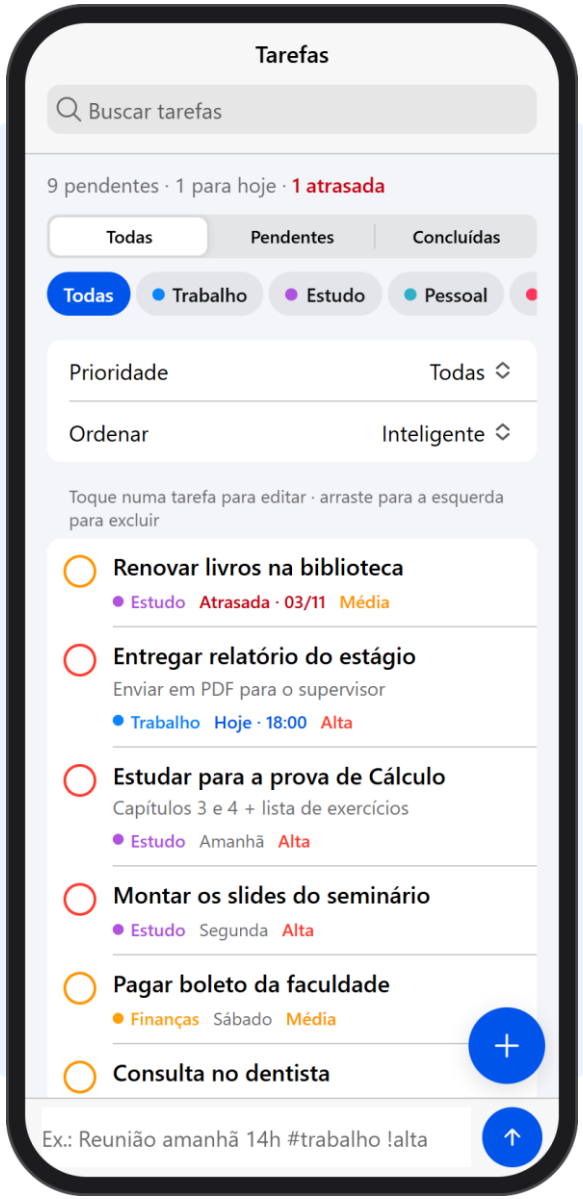
EXPLORANDO TECNOLOGIAS MÓVEIS

Gerenciador de Tarefas Inteligente

Um app para Android, iOS e Web feito com Ionic + Capacitor

Ionic 8 | Capacitor 7 | Vite 6 | JavaScript

Unochapecó — Universidade Comunitária da Região de Chapecó
Professor: Cezar Junior de Souza
Integrantes: _____



ROTEIRO

Roteiro da apresentação

A mesma estrutura do enunciado — de 20 a 30 minutos

A 5 min

Introdução à Tecnologia

- Características
- Prós e contras
- Casos de uso ideais

B 10 min

Processo de Desenvolvimento

- Ambiente e estrutura do projeto
- Componentes de UI
- Estado e dados
- Hot reload ao vivo

C 5 min

Build e Deploy

- Build por plataforma
- APK, IPA e lojas
- Performance

D 5 min

Comparativo com React Native

- Arquitetura
- Fluxo de trabalho
- Quando usar cada um

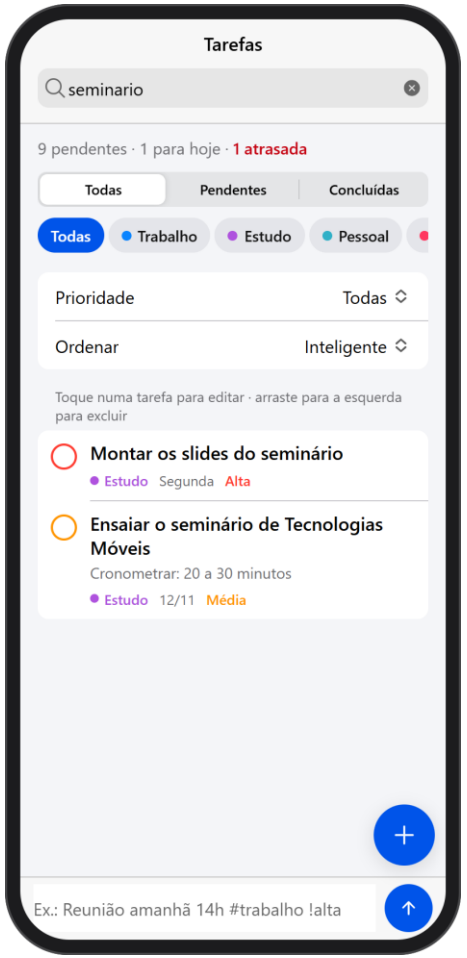
Abertura 2 min · A 5 · B 10 · C 5 · D 5 · Encerramento + perguntas → **≈ 28 min**

O que o enunciado pede — e o app entrega

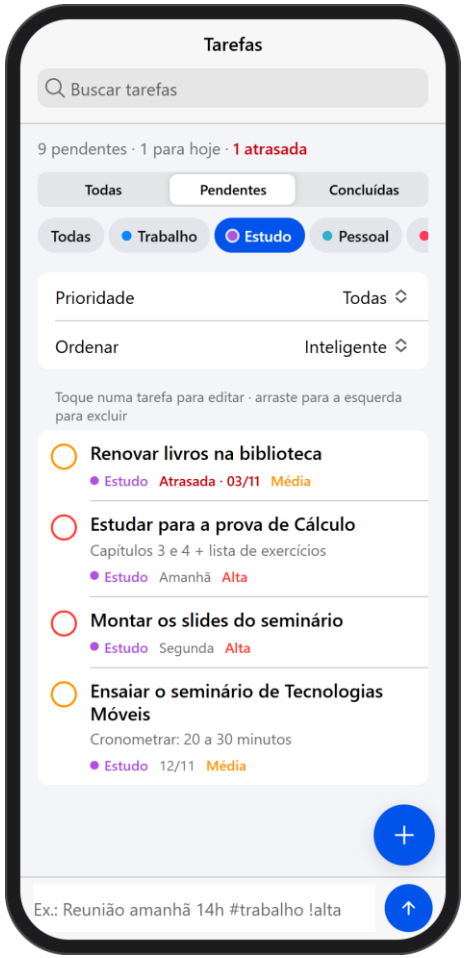
- **Criar, editar e excluir tarefas**
  Formulário em modal · arrastar a tarefa para excluir
- **Categorizar tarefas**
  Trabalho, Estudo, Pessoal, Saúde, Finanças e Outros
- **Definir prioridades**
  Alta, Média e Baixa — com cor própria
- **Marcar como concluídas**
  Checkbox redondo na cor da prioridade
- **Buscar e filtrar**
  Busca sem acento + situação, categoria e prioridade
- **Persistência local**
  Capacitor Preferences no celular · localStorage na web

✨ Além do pedido

Adição rápida em linguagem natural · sugestão automática de categoria e prioridade · ordenação por urgência · modo escuro · abre offline



Busca: “seminario” acha “seminário”



Filtro: Pendentes + Estudo

A

PARTE A

Introdução à Tecnologia

5 minutos

- Características principais
- Vantagens e desvantagens
- Casos de uso ideais

Ionic + Capacitor: como as peças se encaixam



Ionic = a interface

Kit de componentes de UI com aparência e gestos de app. São web components (feitos com Stencil): funcionam com qualquer framework — ou com nenhum.

Capacitor = a ponte nativa

Gera projetos Android e iOS de verdade. Os plugins levam câmera, arquivos, notificações e armazenamento até o JavaScript.

WebView = o motor do navegador

Chromium no Android, WKWebView no iOS. É ela que desenha o HTML e o CSS dentro do app.

Características principais

Open source, licença MIT — Ionic e Capacitor são gratuitos

</>

Web components

Cada componente é uma tag HTML, como <ion-list>. Funciona com Angular, React, Vue — ou com JavaScript puro, como no nosso app (Ionic + Vite).

Visual adaptativo

O mesmo código ganha cara de iOS no iPhone e de Material Design no Android. O modo escuro veio com uma linha de CSS.

1→3

Um código, três plataformas

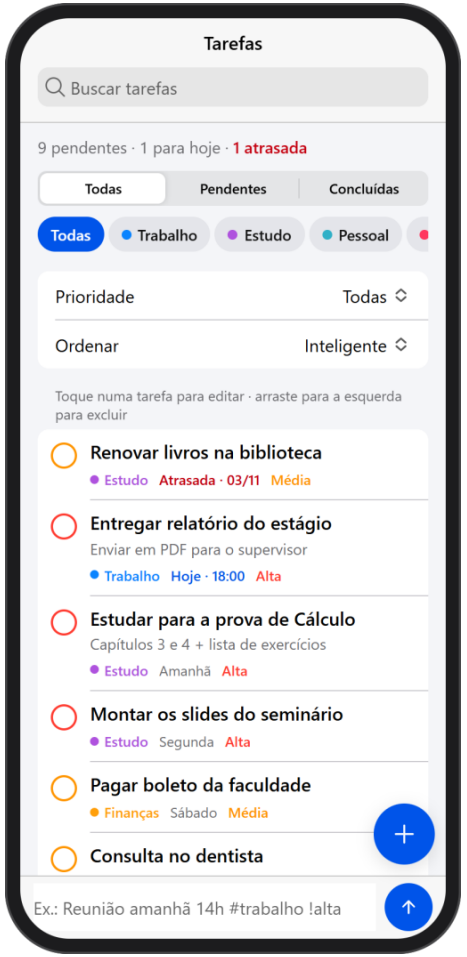
Android, iOS e Web (PWA) saem da mesma pasta dist/ — sem reescrever nenhuma tela.

Acesso nativo por plugins

Plugins do Capacitor chamam APIs do aparelho direto do JavaScript: armazenamento (usamos o Preferences), câmera, notificações, GPS…

Mesmo código, duas aparências

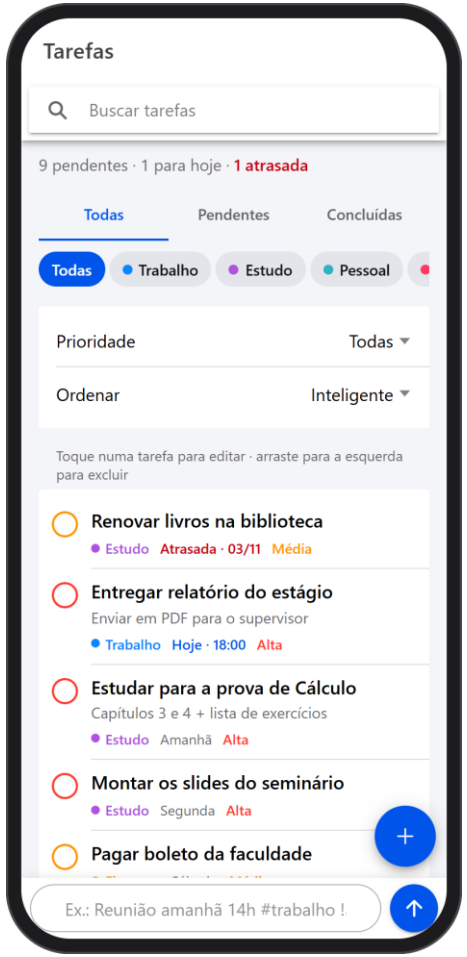
O Ionic detecta a plataforma e aplica o modo ios ou md (Material Design) — sem CSS específico por plataforma



iPhone · modo ios

Componente	iOS	Android
Título (ion-title)	Centralizado	À esquerda
Busca (ion-searchbar)	Campo cinza	Barra com sombra
Situação (ion-segment)	Pílula deslizante	Abas com sublinhado
Seleção (ion-select)	Ícone de setas	Ícone de triângulo
Adição rápida (ion-input)	Campo simples	Contorno arredondado

O styles.css do app é o mesmo nas duas telas



Android · modo md

Vantagens e desvantagens

✓ Vantagens

Reaproveita o conhecimento web
HTML, CSS e JavaScript: curva de aprendizado curta

Um código, várias plataformas
Android, iOS e Web com a mesma base

Ciclo de desenvolvimento rápido
Desenvolve no navegador, com DevTools e hot reload

Ecossistema npm inteiro
Qualquer biblioteca JavaScript funciona

Gratuito e open source
Ionic e Capacitor têm licença MIT

✕ Desvantagens

Roda numa WebView
Animações pesadas, jogos e 3D ficam abaixo do nativo

Visual “quase nativo”
Exige cuidado nos detalhes para parecer app de verdade

Depende de plugins
Sem plugin pronto, é preciso escrever Kotlin ou Swift

Bundle maior
Nosso JavaScript: 659 kB (137 kB com gzip)

iOS ainda exige Mac
O Xcode, que gera o IPA, só roda no macOS

Casos de uso ideais

Onde o Ionic brilha

- ✓ **Apps de listas, cadastros e formulários** — como este
- ✓ **Sistemas internos de empresas**
- ✓ **MVPs e protótipos** que precisam chegar rápido às lojas
- ✓ **Times que já dominam web** (HTML, CSS, JavaScript)
- ✓ **Produtos que também precisam de versão web** (PWA)

Quando evitar

- ✗ Jogos e gráficos 3D
- ✗ Animações complexas e realidade aumentada
- ✗ Processamento pesado (vídeo, áudio)
- ✗ Integrações nativas profundas (widgets)

→ prefira nativo (Kotlin/Swift), Flutter ou React Native

Nosso app — listas, formulários e dados locais — é o caso de uso ideal para Ionic.

B

PARTE B

Demonstração do Processo de Desenvolvimento

10 minutos

- Ambiente e estrutura do projeto
- Componentes de UI mais importantes
- Gerenciamento de estado e dados
- Hot reload ao vivo

Configuração do ambiente

Para desenvolver

Windows, Mac ou Linux

- Node.js 20+ e npm
- VS Code
- Chrome DevTools (modo celular)
- Git e GitHub

Para Android

Funciona no Windows

- Android Studio: SDK, emulador e JDK 21
- Android 6.0 ou mais novo (minSdk 23)

Para iOS

Só no macOS

- Mac com Xcode
- iOS 14 ou mais novo
- Conta Apple Developer para publicar

terminal

```
# criar o projeto (web, JavaScript puro) e instalar Ionic + Capacitor
npm create vite@latest tarefas -- --template vanilla
npm install @ionic/core ionicons @capacitor/core @capacitor/preferences
npm install -D @capacitor/cli
npx cap init "Gerenciador de Tarefas Inteligente" br.com.exemplo.tarefasinteligentes --web-dir dist

# no dia a dia
npm run dev    # servidor com hot reload em http://localhost:5173
npm test       # 27 testes automatizados (node --test)
```

Estrutura de arquivos do projeto

```
DESENWEB/
├─ index.html              as telas: componentes <ion-*>
├─ src/
│  ├─ main.js              interface: eventos e render()
│  ├─ ionic.js             registra componentes, toast e alert
│  ├─ store.js             estado + CRUD + gravação
│  ├─ storage.js           Preferences (celular) / localStorage
│  ├─ smart.js             parser, sugestões, filtros, ordenação
│  ├─ pwa.js               service worker (só na web)
│  └─ styles.css           visual por cima do tema Ionic
├─ public/                 manifest, sw.js (offline), ícones
├─ resources/              ícone e splash dos apps nativos
├─ tests/                  27 testes (node --test)
├─ android/  ios/          projetos nativos (npx cap add)
├─ slides/                 esta apresentação
├─ capacitor.config.json   appId, nome e webDir: "dist"
├─ vite.config.js          servidor dev (porta 5173) + build
└─ package.json            scripts: dev, build, test, android:apk
```

index.html = as telas

Os componentes <ion-*> ficam no HTML; o main.js só liga os eventos e redesenha a lista.

Regras separadas da tela

smart.js e store.js não usam o DOM: são testados no Node, sem navegador (27 testes).

android/ e ios/ são gerados

Projetos nativos reais criados pelo npx cap add; o npx cap sync copia o build para dentro deles.

Componentes de UI mais importantes

1. `ion-header · ion-toolbar · ion-title`
   Barra superior fixa com o título

2. `ion-searchbar`
   Busca que ignora acentos e maiúsculas

3. `ion-segment`
   Situação: todas, pendentes ou concluídas

4. `ion-chip`
   Filtro por categoria (rola na horizontal)

5. `ion-select + ion-popover`
   Prioridade e ordenação em menu flutuante



6. `ion-list · ion-item-sliding · ion-label`
   Cada tarefa: toque edita, arrastar exclui

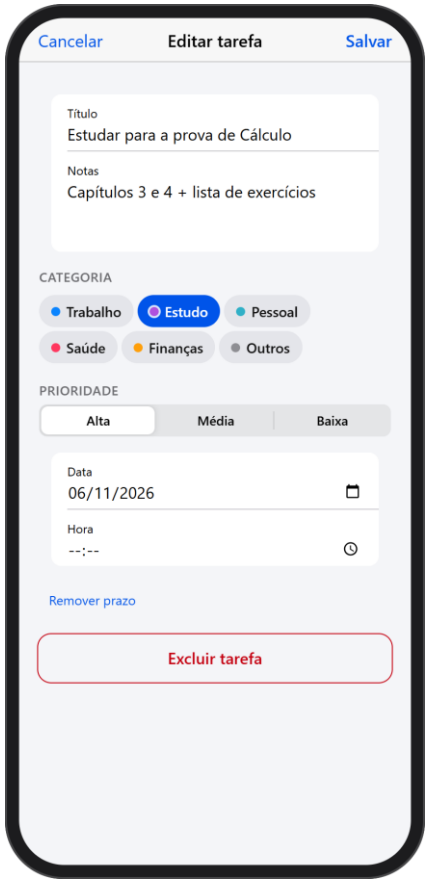
7. `ion-checkbox`
   Concluir — o anel tem a cor da prioridade

8. `ion-fab`
   Botão flutuante “nova tarefa” (abre o ion-modal)

9. `ion-footer + ion-input`
   Adição rápida em linguagem natural

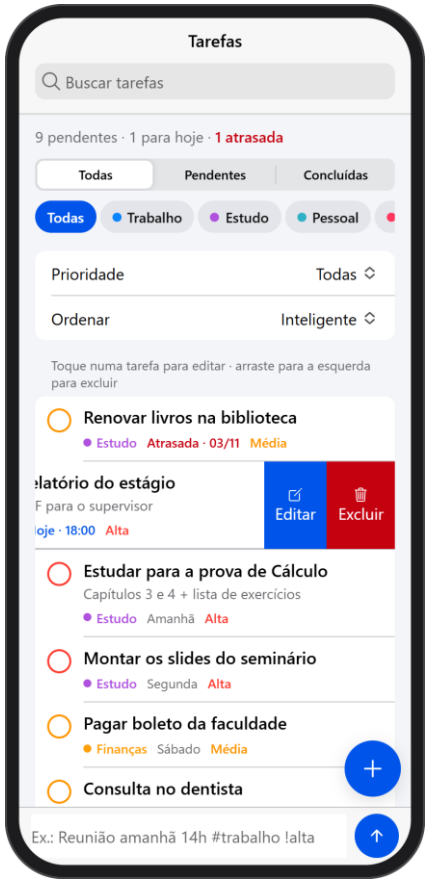
Também usados: ion-modal, ion-toast, ion-alert, ion-icon e ion-content (a área que rola) — próximo slide.

Componentes em ação



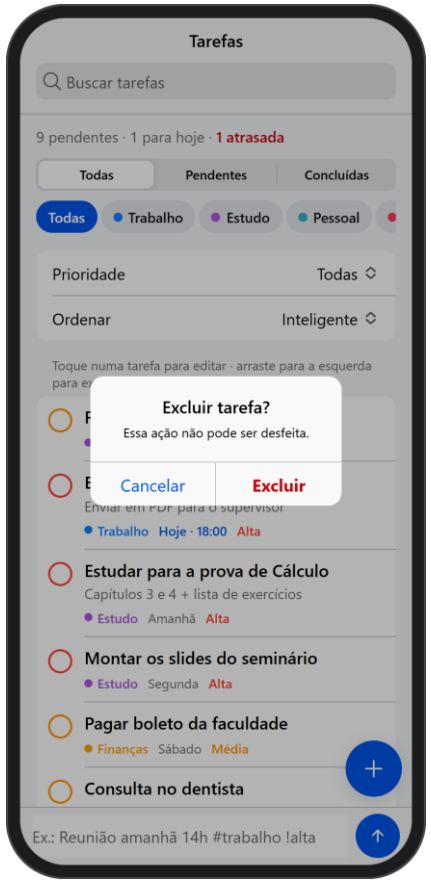
`ion-modal`

Formulário de criar/editar: ion-input, ion-textarea e ion-segment



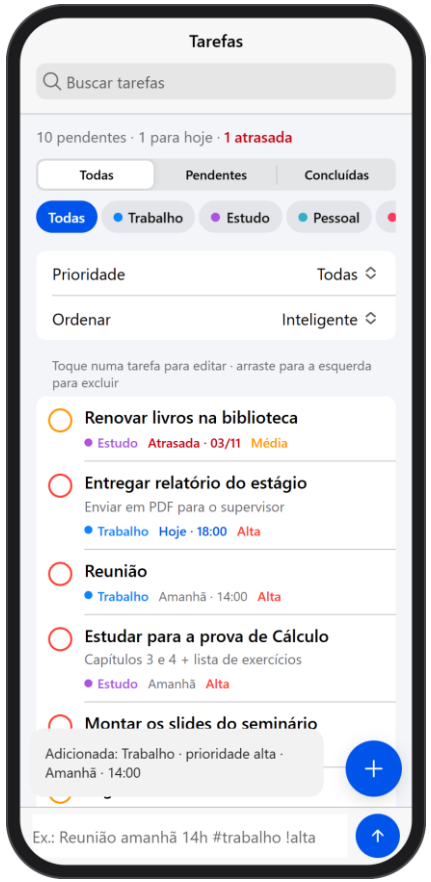
`ion-item-sliding`

Arrastar para a esquerda revela Editar e Excluir



`ion-alert`

Confirma antes de excluir



`ion-toast`

Aviso rápido — em vermelho se falhar ao salvar

Um componente Ionic no código: o HTML declara, o JS reage

index.html — botão flutuante

```html
<ion-fab slot="fixed" vertical="bottom" horizontal="end">
  <ion-fab-button id="btn-new" aria-label="Nova tarefa (detalhada)">
    <ion-icon name="add" aria-hidden="true"></ion-icon>
  </ion-fab-button>
</ion-fab>
```

index.html — formulário (resumido)

```html
<ion-modal id="sheet" aria-labelledby="sheet-title">
  <ion-header>…</ion-header>
  <ion-content class="ion-padding">
    <form id="task-form" novalidate autocomplete="off">…</form>
  </ion-content>
</ion-modal>
```

src/main.js — trechos

```js
$('btn-new').addEventListener('click', () => openSheet());

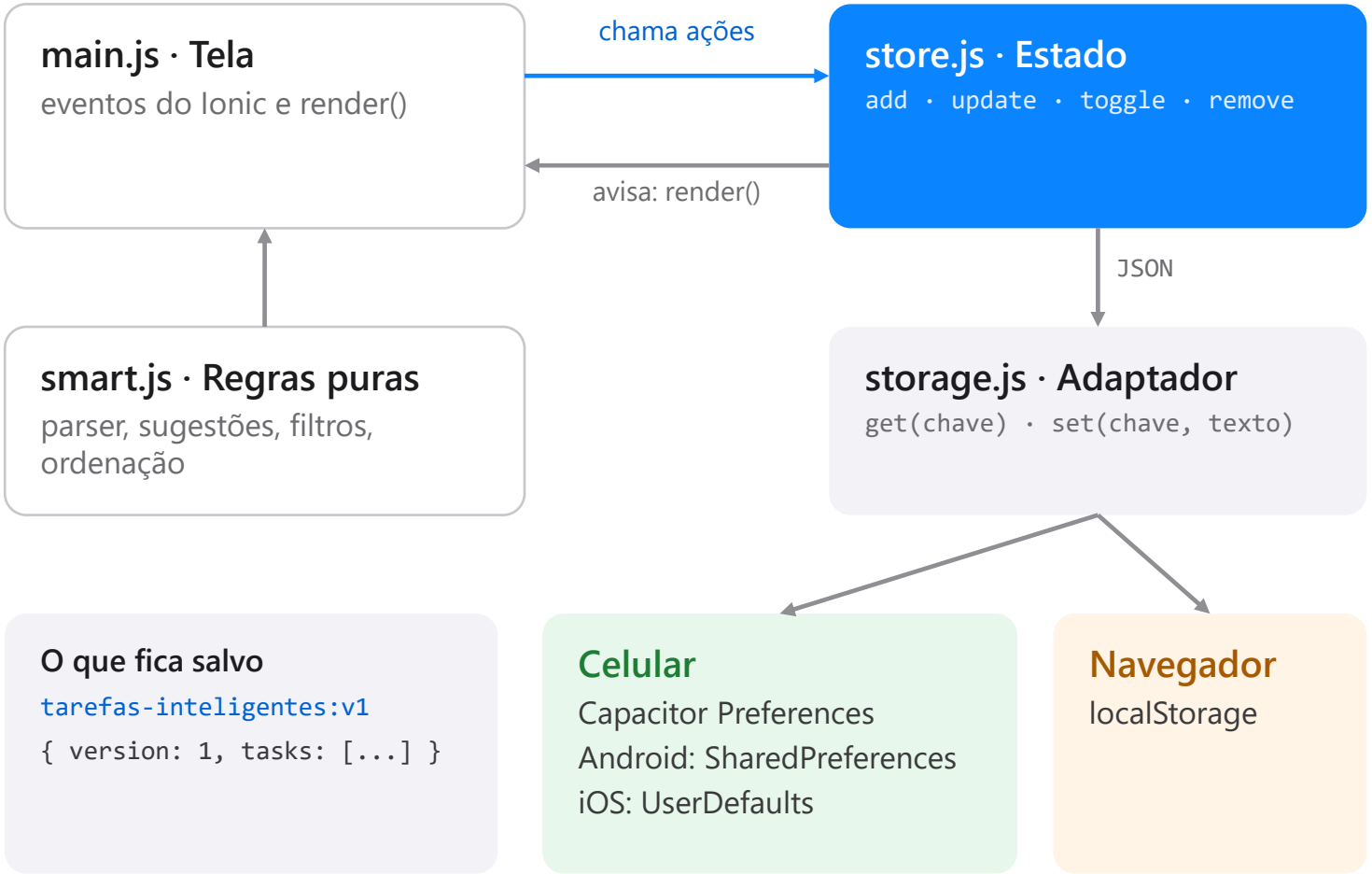
async function openSheet(task = null) {
  // …
  await $('sheet').present();
  if (!task) $('t-title').setFocus();
}

$('status').addEventListener('ionChange', (e) => {
  filters.status = e.detail.value;
  render();
});
```

`src/ionic.js` registra os 32 componentes usados (defineCustomElement) — só eles entram no bundle.

Gerenciamento de estado e dados

Sem biblioteca de estado: um store simples com assinatura (padrão observer)



Uma ação, passo a passo

1. Usuário marca, cria ou exclui uma tarefa.
2. O store muda a lista em memória e avisa os assinantes.
3. render() redesenha na hora — sem esperar o disco.
4. Grava em fila: { version: 1, tasks: [...] }
5. Se a gravação falhar: desfaz e avisa com um toast vermelho.

store.js por dentro: tela rápida, dado seguro

src/store.js — linhas 49 a 68

```
const commit = (change) => {
  pending.push(change);
  tasks = change(tasks);                                                    1
  emit();
  const run = writing.then(async () => {                                    2
    const next = change(saved);
    try {
      await storage.set(STORAGE_KEY, JSON.stringify({ version: 1, tasks: next }));
    } catch (e) {                                                           3
      pending.splice(pending.indexOf(change), 1);
      tasks = pending.reduce((list, fn) => fn(list), saved);
      emit();
      throw new Error('Não foi possível salvar a alteração. Tente novamente.', { cause: e });
    }
    saved = next;
    pending.splice(pending.indexOf(change), 1);
  });
  writing = run.catch(() => {}); // uma falha não trava a fila               4
  return run;
};
```

1. Otimista: muda a memória e redesenha a tela na hora.
2. Grava em fila (writing.then) — nunca fora de ordem.
3. Falhou? Desfaz a mudança e rejeita com Error → o app mostra o toast “Não foi possível salvar”.
4. A fila continua andando mesmo depois de um erro.

✓ Coberto por testes
“falha ao salvar: add rejeita, desfaz a mudança e avisa os assinantes”

O “inteligente”: smart.js entende texto livre

ADIÇÃO RÁPIDA (rodapé do app)

Reunião **amanhã 14h** **#trabalho** **!alta**

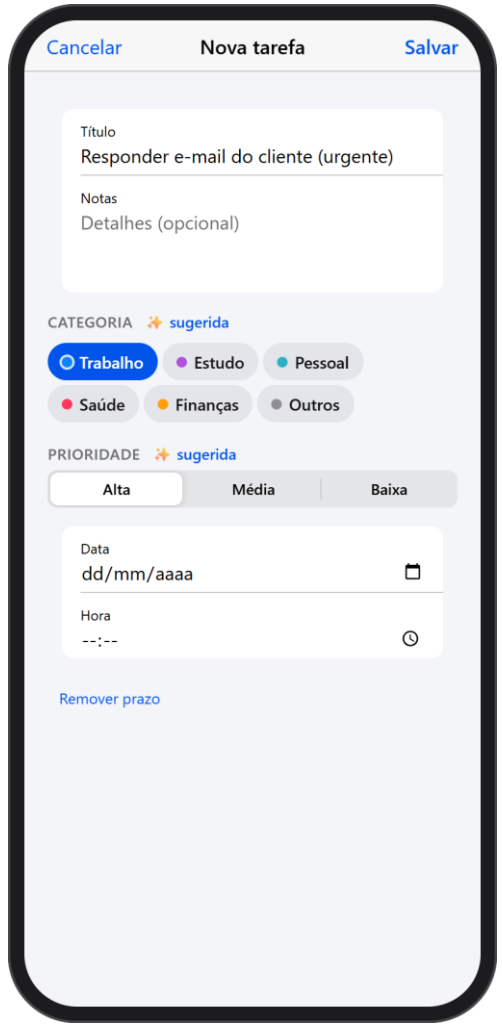
título: Reunião | ● Trabalho | prioridade Alta | Amanhã · 14:00

SUGESTÃO NO FORMULÁRIO (sem tags)

Responder **e-mail** do **cliente** (**urgente**)

✨ Trabalho (e-mail, cliente) | ✨ Alta (urgente)

Entende: hoje, amanhã, sexta, 13/11, em 3 dias, 14h, 14h30, às 8h
Não se confunde: “Comprar pão 2h” é duração e “1/2” é fração



Hot reload ao vivo com o Vite

1. **Salvar o arquivo**
   Ctrl + S no VS Code
2. **O Vite envia só o que mudou**
   pelo WebSocket, sem novo build
3. **A tela atualiza**
   em menos de 1 segundo

Arquivo	O que mudar na demo	O que acontece
`src/styles.css`	`--ion-color-primary: #0a84ff → #af52de`	A cor muda sem recarregar — o formulário aberto continua aberto
`index.html`	`<ion-title>Tarefas</ion-title> → Minhas tarefas`	A página recarrega sozinha; as tarefas continuam salvas
`src/smart.js`	`+ { id: 'lazer', label: 'Lazer', … }`	Recarrega e o chip “Lazer” aparece nos filtros e no formulário

terminal

```
$ npm run dev
  VITE v6.4.5  ready in … ms
  ➜  Local:   http://localhost:5173/
  ➜  Network: http://192.168.x.x:5173/
```

No celular, ao vivo

Mesma rede Wi-Fi: abra o endereço Network (server.host: true no vite.config.js). Dentro do app nativo:

```
npx cap run android -l --host <IP> --port 5173
```

C

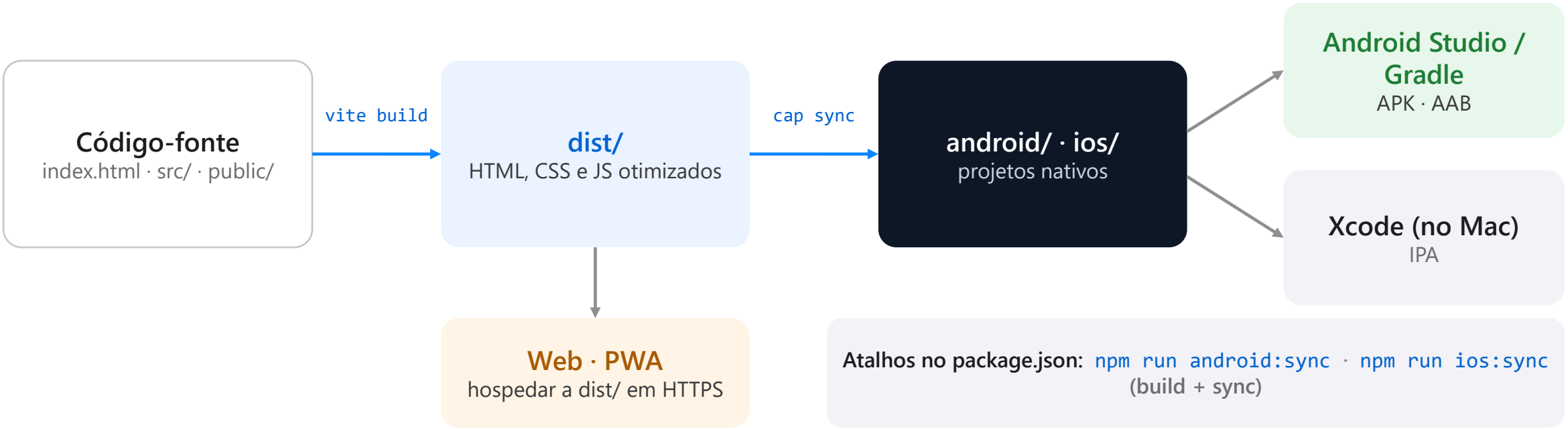
PARTE C

Build e Deploy

5 minutos

- Processo de build para diferentes plataformas
- Geração de APK/IPA e publicação nas lojas
- Considerações sobre performance

Do código ao app: o pipeline de build



terminal

```
npm run build          # vite build → dist/   (≈ 3 s, 112 módulos)
npx cap add android    # só na 1ª vez (idem: npx cap add ios)
npx cap sync           # copia a dist/ e os plugins para android/ e ios/
npx cap open android   # abre no Android Studio (ios → Xcode)
```

Gerando APK / IPA e publicando nas lojas

Android

TESTE (DEBUG)

```
npm run android:apk
```

- Faz build + cap sync + gradlew assembleDebug
- Gera android/app/build/outputs/apk/debug/app-debug.apk
- Instala copiando para o celular ou com adb install

LOJA (RELEASE)

- Android Studio › Build › Generate Signed App Bundle / APK
- Assina com uma keystore (guarde-a!) → arquivo .aab
- Google Play Console — taxa única de US$ 25

iOS

REQUISITO: MAC COM XCODE

```
npm run ios:sync
```

- npx cap open ios → abre o projeto no Xcode
- Signing & Capabilities: Team e Bundle ID
- Product › Archive → IPA → TestFlight / App Store

CUSTOS E ALTERNATIVAS

- Apple Developer Program: US$ 99 por ano
- Sem Mac: build na nuvem com macOS (GitHub Actions, Codemagic)

Web / PWA: `npm run build` → publicar a dist/ em HTTPS (GitHub Pages, Netlify…) → "Adicionar à tela inicial" — abre offline após a 1ª visita

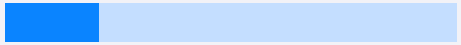
Considerações de performance

WebView × nativo

A tela é HTML e CSS desenhados pela WebView. Para listas e formulários fica fluido; jogos, 3D e animações pesadas ficam abaixo do nativo.

Tamanho do bundle

JS



659,18 kB → 136,52 kB gzip

CSS

48,46 kB → 8,33 kB gzip

⚠ O Vite avisa: pedaço > 500 kB — é o Ionic

Offline e dados locais

No APK/IPA, os arquivos já vão dentro do app: abre sem internet. Na web, o service worker pré-cacheia o build. Os dados ficam no aparelho.

npm run build — saída real

```
dist/assets/index-….css    48.46 kB │ gzip:   8.33 kB
dist/assets/index-….js    659.18 kB │ gzip: 136.52 kB
(!) Some chunks are larger than 500 kB after minification.
✓ built in 3.23s
```

O que já fazemos

- ✓ Só os 32 componentes usados entram
- ✓ Preferences carregado sob demanda
- → Dá para separar o Ionic (manualChunks)

D

PARTE D

Comparativo com React Native

5 minutos

- Como cada um desenha a tela
- Diferenças no fluxo de desenvolvimento
- Quando escolher cada um

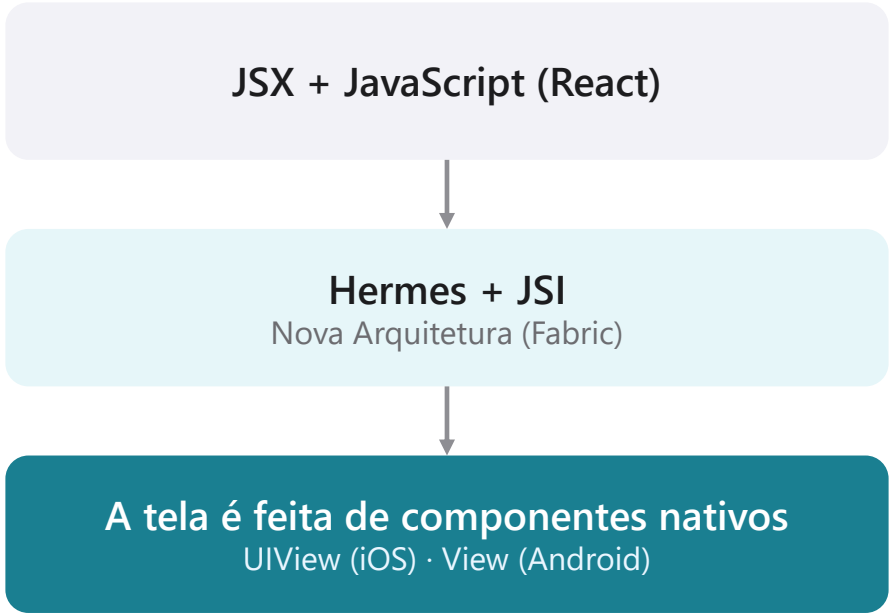
Como cada um desenha a tela

Ionic + Capacitor



Recursos do aparelho: plugins do Capacitor

React Native



Recursos do aparelho: módulos nativos ou Expo SDK

Diferenças no fluxo de desenvolvimento

Etapa	Ionic + Capacitor (nosso app)	React Native (visto em aula)
Criar o projeto	npm create vite + @ionic/core (ou ionic start)	npx create-expo-app
Escrever a tela	HTML + CSS de verdade; JS puro ou qualquer framework	JSX + StyleSheet (subconjunto de CSS em objetos JS)
Rodar e testar	No navegador (npm run dev), DevTools em modo celular	Expo Go no celular ou emulador, via Metro
Hot reload	Vite HMR: CSS sem recarregar; JS recarrega a página	Fast Refresh: mantém o estado do componente
Depurar	DevTools do navegador (chrome://inspect, Safari)	React Native DevTools
Recurso nativo	Plugin Capacitor (ex.: @capacitor/preferences)	Módulo nativo ou Expo SDK
Gerar o app	vite build → cap sync → Android Studio / Xcode	EAS Build (nuvem) ou Gradle / Xcode
Versão web	O mesmo código já é site/PWA	Via react-native-web (Expo)

Resumo: no Ionic o ciclo é o da web; no React Native, o de um app nativo escrito em JavaScript.

A mesma tela nas duas tecnologias

Ionic — HTML (simplificado de src/main.js)

```html
<ion-list>
  <ion-item-sliding>
    <ion-item>
      <ion-checkbox slot="start"></ion-checkbox>
      <ion-label>Estudar para a prova</ion-label>
    </ion-item>
    <ion-item-options side="end">
      <ion-item-option color="danger">Excluir</ion-item-option>
    </ion-item-options>
  </ion-item-sliding>
</ion-list>
```

React Native — JSX (equivalente)

```jsx
<FlatList
  data={tarefas}
  renderItem={({ item }) => (
    <Swipeable renderRightActions={BotaoExcluir}>
      <View style={styles.linha}>
        <Checkbox value={item.done} />
        <Text>{item.title}</Text>
      </View>
    </Swipeable>
  )}
/>
```

EQUIVALÊNCIAS

ion-list → FlatList	ion-item-sliding → Swipeable	ion-checkbox → expo-checkbox	ion-modal → Modal
ion-alert → Alert.alert	ion-searchbar → TextInput	CSS → StyleSheet	Preferences → AsyncStorage

Quando escolher cada um

Escolha Ionic + Capacitor quando...

- A equipe já domina HTML, CSS e JavaScript
- O app é de formulários, listas e dados
- Também precisa de versão web (PWA)
- O prazo é curto e um código só ajuda

Escolha React Native quando...

- A sensação 100% nativa é prioridade
- Há animações e gestos intensos
- A equipe já trabalha com React
- Quer o ecossistema Expo (EAS na nuvem)

Nossa conclusão

Para um app de tarefas — listas, formulários e dados locais —, o Ionic entregou Android, iOS e Web com um único código HTML/CSS/JS, e quase todo o desenvolvimento aconteceu no navegador.

Obrigado!

Perguntas?

Código-fonte e slides no GitHub

github.com/jdecarli6/DESENWEB

3

plataformas: Android, iOS e Web

32

componentes Ionic

27

testes automatizados

